

Lab Manual 7

Lab Title

Implement Graph Search Algorithms like BFS and DFS

Objective

To implement and understand the working of **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** algorithms on a graph using Python.

Introduction

Breadth-First Search (BFS)

Breadth-First Search (BFS) is a graph traversal algorithm that explores all the vertices of a graph or tree layer by layer. It uses a queue to keep track of nodes to visit next. BFS is commonly used in **AI, pathfinding algorithms, and network broadcasting**.

Depth-First Search (DFS):

Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking. It uses a **stack** (either explicitly or through recursion) and is widely used in **AI problem-solving, game trees, pathfinding, and cycle detection**.

Theoretical Background

1. Graph Basics:

1. A graph is a collection of **nodes (vertices)** and **edges (connections)**.
2. Can be **directed or undirected**.
3. Can be **cyclic or acyclic**.

2. Breadth-First Search (BFS):

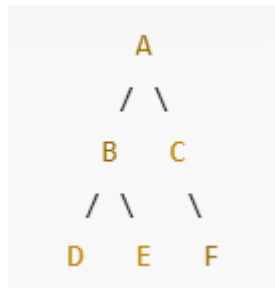
1. Explores neighbors level by level.
2. Uses a **queue** (FIFO).
3. Suitable for finding the **shortest path** in unweighted graphs.

3. Depth-First Search (DFS):

1. Explores as **deep as possible** before backtracking.
2. Uses a **stack** or recursion (LIFO).
3. Useful for exploring entire components or **checking connectivity**

➤ Breadth-First Search (BFS):

Graph Example :



Adjacency List Representation:

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': [],  
    'F': []  
}
```

Pseudocode for BFS:

BFS(Graph, Start_Node):

Initialize an empty queue

Initialize an empty set for visited nodes

Enqueue Start_Node into queue

While queue is not empty:

Dequeue a node from queue → current_node

If current_node is not visited:

Visit the node (e.g., print or store)

Mark current_node as visited

Enqueue all unvisited neighbors of current_node into queue

➤ BFS Algorithm Breakdown

1. Initialize an empty queue

Think of a **queue** as a line where nodes will be processed. You start with no nodes in the queue.

2. Initialize an empty set for visited nodes

You need to keep track of the nodes you've already visited to avoid going in circles.

3. Enqueue the Start_Node into the queue

You put the starting node (where you begin the search) into the queue, because BFS works by processing nodes in order.

4. While queue is not empty

As long as there are nodes in the queue, you keep going. The queue ensures that nodes are processed level-by-level.

5. Dequeue a node from the queue → current_node

Take the first node from the front of the queue to process it.

6. If current_node is not visited

Check if the current node has already been processed. If not, continue.

7. Visit the node (e.g., print or store)

"Visiting" means doing something with the node, like printing it or storing it in a list.

8. Mark current_node as visited

Now that you've visited the node, mark it to avoid revisiting it later.

9. Enqueue all unvisited neighbors of current_node into queue

Look at all the neighbors (connected nodes) of the current node. If they haven't been visited, add them to the queue to process later.

Python Implementation:

```
from collections import deque
# Step 1: Define kran ga graph ko adjacency list use kr ka
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [ ],
    'E': [ ],
    'F': [ ]
}
# Step 2:
def bfs(start_node):
    visited = set() # To keep track of visited nodes
    queue = deque([start_node]) # Initialize kr rha queue ko starting node sa
```

```

print("BFS Traversal:")
while queue:
    current = queue.popleft() # Dequeue kra ga front of queue sa start sa
    if current not in visited:
        print(current, end=" ") # Visit kran ga node ko
        visited.add(current) # Mark kr da ga ka ya node visit ho gya
        # Enqueue kra ga unvisited neighbors ko
        for neighbor in graph[current]:
            if neighbor not in visited:
                queue.append(neighbor)
# Step 3: Call kra ga BFS function with starting node
bfs('A')

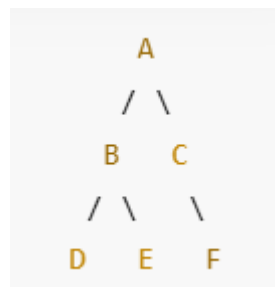
```

Output:

BFS Traversal:
A B C D E F

➤ Depth-First Search (DFS):

Graph Example :



Adjacency List Representation:

```

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

```

Pseudocode for DFS:

DFS(Graph, Start_Node):

Initialize an empty set for visited nodes

Call DFS_Recursive(Graph, Start_Node, visited)

DFS_Recursive(Graph, Node, Visited):

```
If Node is not in Visited:
    Visit the node (e.g., print or store)
    Add Node to Visited
    For each neighbor of Node in Graph:
        Recursively call DFS_Recursive on neighbor
```

Explanation in Simple Words:

Line 1 → DFS(Graph, Start_Node):

This starts the DFS function. It receives the graph and the starting node.

Line 2 → Initialize an empty set for visited nodes

We create an empty set named 'visited' to keep track of nodes we have already visited.

Line 3 → Call DFS_Recursive(Graph, Start_Node, visited)

We start the recursive DFS process by calling the DFS_Recursive function.

Line 4 → DFS_Recursive(Graph, Node, Visited):

This is the recursive function that will be repeatedly called for each neighbor.

Line 5 → If Node is not in Visited:

We check if the node has already been visited. If not, we continue.

Line 6 → Visit the node (print or store)

We visit the node. This means printing it or storing it to show DFS order.

Line 7 → Add Node to Visited

We add the current node to the visited set so we don't visit it again.

Line 8 → For each neighbor of Node in Graph:

We look at all the nodes that are directly connected (neighbors).

Line 9 → Recursively call DFS_Recursive on neighbor

For each neighbor, we call DFS_Recursive again. This is how DFS goes deep.

Python Implementation:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

# DFS function define kar rahay hain
def dfs(node, visited=None):
    # visited set woh jagah hai jahan hum pehle hi ja chuke hain
```

```
    if visited is None:
        visited = set()    # Agar visited set nahi bana hua toh naya
set banao
    # Yeh check karta hai ke hum is node par pehle aaye hain ya nahi
    if node not in visited:
        print(node, end=" ") # Node ko print karo (yaani "visit" karo)
        visited.add(node)    # Node ko visited set mein daal do
        # Ab iss node ke tamam neighbors ko visit karne jayenge
        for neighbor in graph[node]:
            # Har neighbor par DFS ko phir se call karenge (recursion)
            dfs(neighbor, visited)
# Yahan se DFS traversal start hota hai
print("DFS Traversal:")
dfs('A')    # DFS start ho raha hai node A se
```

Output:

DFS Traversal: A B D E C F